micro-os-plus-iii-cortexm

The Cortex-M port of µOS++ III — the third architecture project, beside micro-os-plus-iii-aarch32 and micro-os-plus-iii-aarch64, built the same way and from the same kernel.

What is here

```
micro-os-plus-iii-cortexm/
├── CMakeLists.txt
├── include/cmsis-plus/rtos/port/    the port contract the kernel
includes
├── include-m33/                     the generic Cortex-M33 port
headers
├── include-rp2350/                  the RP2350 SMP port headers
├── src/rtos/os-core.cpp             upstream: PendSV, SysTick,
critical sections
├── src/rtos/os-core-m33.cpp         the generic M33 core (SSE-200
CPUID/MHU)
├── src/rtos/os-core-rp2350.cpp      the RP2350 SMP core (SIO
lock/FIFO/CPUID)
├── src/semihosting-exit.cpp         strong _Exit() through
SYS_EXIT
├── src/libc/getentropy.c            getentropy() for libstdc++
└── test/
    ├── CMakeLists.txt  hw.sh  qemu.sh
    ├── boards/<id>/                 board.cmake, include/, src/,
linker.ld,
    │                                openocd.cfg, hw.sh (qemu.sh
where modelled)
    └── <id>/                        that board's test
applications
```

Six boards, two Cortex-M profiles, one and two CPUs:

board	part	CPUs	applications
nucleof411	STM32F411RE, Cortex-M4F	1	mos-test1, cmsis-os-validator, mutex-stress, rtos-apis
weactf411	STM32F411CE, Cortex-M4F	1	mos-test1, spi-pipeline,

board	part	CPUs	applications
`weactf412`	STM32F412RE, Cortex-M4F	1	`cmsis-os-validator`, `mutex-stress`, `rtos-apis`
`pico2`	RP2350, 2× Cortex-M33	**2**	`mos-test1`, `uart-test1`, `cmsis-os-validator`, `mutex-stress`, `rtos-apis`
`pico2-pizero`	RP2350B, 2× Cortex-M33	**2**	`smp-test0`…`smp-test5`, `smp-mat-test`, `sc-test-ko`, `fp-switch`, `exc-test`, the USB tests, `cmsis-os-validator`, `mutex-stress`, `rtos-apis`, `cmsis-os-validator-ram`, `mutex-stress-ram`, `rtos-apis-ram`, `smp-mat-test-ram`
`pico2-rp2350b-psram`	RP2350B + 8 MB PSRAM, 2× Cortex-M33	**2**	the `pico2` hardware set (no PSRAM)
			the `pico2` set plus the PSRAM tests

Building

Clone the two siblings beside this one, then:

```
cmake -S . -B test/build -G Ninja -DBOARD=nucleof411 \
      -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp.git/cmake/
        toolchains/arm-none-eabi.cmake
cmake --build test/build -j
```

Running

Hardware only for the three STM32 boards: they are programmed over an ST-Link and run on the desk. The three RP2350 boards are programmed over CMSIS-DAP. OpenOCD does the whole job: program, enable semihosting, reset run. There is no GDB in this path and nothing is redirected.

```
BOARD=nucleof411 ./test/hw.sh          # list what is built
BOARD=nucleof411 ./test/hw.sh mos-test1
```

`test/qemu.sh` is a dispatcher to `test/boards/<id>/qemu.sh`. Only `pico2` sets `UOS_BOARD_LINKER_QEMU`, so it builds an emulated image per test as well as the hardware one; the STM32 boards and `pico2-pizero` build hardware images only, and the dispatcher says so.

Pure RAM Execution on Pico 2 (RP2350)

Four tests run **entirely from internal SRAM** (`0x20000000`, 520 KB) without writing anything to external QSPI flash: - `cmsis-os-validator-ram` - `mutex-stress-ram` - `rtos-apis-ram` - `smp-mat-test-ram` (dual-core SMP parallel block linear solver)

Each test directory (`test/pico2/<app>/`) contains: - `linker-ram.ld`: Links `.text`, `.rodata`, `.data` in SRAM `0x20000000`. Sections `.bss`, `.heap`, `.stack` are marked `(NOLOAD)` and `.embedded_end_block` is embedded compactly in `.text` so the downloaded ELF size remains compact (~26–68 KB). - `hw.sh`: Dedicated hardware test runner that: 1. Relocates OpenOCD's target work-area to `0x20070000` (`rp2350.cm0 configure -work-area-phys 0x20070000 -work-area-size 0x8000`) so fast CRC verification never overlaps with application memory. 2. Downloads the ELF into SRAM `0x20000000` via `load_image` and validates via `verify_image`. 3. Resumes Core 1 (`rp2350.cm1`) to wait in bootrom. 4. Configures Core 0 (`rp2350.cm0`) VTOR (`0xE000ED08`) to `0x20000000`, sets SP from `[0x20000000]` and PC from `[0x20000004] & ~1` (`_reset_handler`), enables semihosting, and resumes. 5. Monitors semihosting output for `RESULT: PASS` and exits cleanly. - `load-ram.cfg`: Standalone OpenOCD script for manual and interactive debug sessions. - `README.md` and `README.pdf`: Self-contained test documentation and architecture notes.

Generic QEMU cores

Besides the board cores, the port exports generic single-core QEMU cores that the harness's emulated configurations link, in `tests/device-qemu-cortexm` (vector table, CMSIS core, linker script):

target	CPU	used by
micro-os-plus::cortexm-qemu-m0	Cortex-M0 (-mfloat-abi=soft)	qemu-cortex-m0
micro-os-plus::cortexm-qemu-m3	Cortex-M3 (-mfloat-abi=soft)	qemu-cortex-m3
micro-os-plus::cortexm-qemu-m4f	Cortex-M4F (-mfpu=fpv4-sp-d16)	qemu-cortex-m4f
micro-os-plus::cortexm-qemu-m7	Cortex-M7F (-mfpu=fpv5-d16)	qemu-cortex-m7f
micro-os-plus::cortexm-qemu-m33	Cortex-M33 (-mfpu=fpv5-sp-d16), 1 or 2 CPUs	pico2-1cpu, 2xcortex-m33

micro-os-plus::cortexm-qemu is kept as an alias of the M7 core. Each core shares src/rtos/os-core.cpp (or os-core-m33.cpp), src/semihosting-exit.cpp and the generic include/, and differs only in the -mcpu flags. The harness platform sets OS_NCPU and, for the SMP M33 case, OS_USE_SMP_SCHEDULER; the qemu-cortex-* platforms run the SMP kernel single-core at OS_NCPU=1.

SMP, and where it lives

pico2 is the SMP board. Two identical, cache-coherent Cortex-M33 in one coherency domain — unlike the RK3506's Cortex-M0 companion, these two are peers, so the scheduler places threads on both and either can take an IPI from the other.

Three pieces of RP2350 silicon make that work, and all three are the **board's**, not the port's:

kernel lock	SIO hardware spinlock 0 — not LDREX/STREX; the RP2350 has no global exclusive monitor, so the architectural route does not work across the two cores
IPI	the SIO inter-core FIFO on SIO_IRQ_FIFO (external IRQ 25); the handler drains the FIFO and pends its own core's PendSV
CPU index	SIO_CPUID

Core 1 is launched through the bootrom FIFO handshake with its own MSP. Only core 0 advances the RTOS clock; core 1's SysTick reschedules core 1 alone.

The SIO spinlocks survive a debugger reset and a re-flash, so the board's `boot.S` releases all 32 on core 0 at reset: a previous run that ended holding spinlock 0 would otherwise hang the next image at its first critical section.

Each thread keeps its own FPU registers, on either core: FPCCR.ASPEN and LSPEN are on (lazy stacking of s0–s15 and FPSCR on exception entry), and PendSV saves s16–s31 for a thread whose frame is extended. The suite `fp-switch` checks it.

One CPU, and why that matters

`src/rtos/os-core.cpp` is upstream's Cortex-M port, carried over unmodified. It is single-core, and on these three boards that is what runs: `OS_NCPU=1`, the kernel's SMP branches compile out, and **no kernel patch is applied**. That is exactly how the predecessor proved these boards, and it is what makes them the witness that the SMP port contract is optional rather than assumed.

`pico2` uses a different core — `src/rtos/os-core-rp2350.cpp`, with the port headers that differ in `include-rp2350/`, searched before `include/` — and names it in its `board.cmake` (`UOS_BOARD_PORT_CORE`, `UOS_BOARD_PORT_INCLUDE`). The two are 91% the same file: pico2's is upstream's plus an `OS_USE_SMP_SCHEDULER` branch.

They are not merged today, for a stated reason rather than an oversight. The three STM32 boards are hardware-proven on upstream's core, and moving them onto a different 748-line scheduler has to be re-proven on the desk, not assumed. A board therefore names the core it was proven with, and adding `pico2` reached no STM32 board. Merging them is a later step.